Save Point

Future Work

AlotaNotes Quiz App · Prepared for Logan Young, Save Point Limited · 27 October 2026

Known, intentional deferrals and gaps, not things discovered late and hidden. The planning document this project grew from (`Docs/flutter_rewrite_plan.md`, moved here from the Python repo) has the full history of decisions; this file is the current, standalone summary.

Blocked on things only a human can provide

Not attempted, not faked with a stub that pretends to work:

- **Checking reminders on real devices.** Study reminders are built (`services/reminder_service.dart`, Settings > Reminders) and tested against a fake notification plugin, but delivery hasn't been seen on a real phone or desktop yet. Worth checking: the Android 13+ permission prompt, an inexact scheduled notification arriving with the app closed and after a reboot, and the Linux/Windows while-open reminder. Windows only shows reminders while the app is open because it can't cancel scheduled toasts without MSIX packaging; packaging it as MSIX would allow real scheduling there.
- **Cloud storage integrations** (Dropbox, Google Drive, OneNote). Each needs a real developer-console app registration (client ID/secret) that only the project owner can create. Building OAuth flow scaffolding against nonexistent credentials would be unverifiable busywork, so nothing has been built here yet, not even a placeholder screen.
- **Mobile app store submission** (Apple App Store, Google Play). Needs an Apple Developer account (plus a Mac, for iOS) and a Google Play Console account; inherently a human/billing decision, not something to attempt autonomously. **Desktop distribution is different and is built**: `.github/workflows/build.yml` produces a Windows installer and a Linux AppImage on every GitHub release. See `packaging.md`, including the honest caveat that neither build has been run end-to-end yet (no Windows machine and no root for the Linux toolchain were available while writing it).

Deliberate design choices, not gaps

- **No real accounts or backend.** See `lib/auth/auth_service.dart`'s doc comment. `HardcodedAuthService` (a single local admin/1234 session) is meant to be swapped for a real provider later without touching any screen, the same reasoning as the Python desktop app's swappable `ApiKeyStore`.
- **No cross-device sync.** Falls out of the no-backend decision above.
- **Bulk import stages a flat, reorderable list into one destination folder, not a reconstructed source folder tree.** Standard mobile file pickers return individually-picked files, not an enumerated folder tree; there's no reliable cross-platform "pick a whole folder recursively" API the way a desktop file dialog

has. See `bulk_import_staging_screen.dart`'s own doc comment. Revisit if a real need for nested-source import shows up.

- **No automatic note-linking / keyword extraction.** The Python desktop app cross-links notes that share concepts via local keyword matching, and feeds linked-note context into AI question generation. Not ported: this app's `LinkedNoteContext` type exists in `ai/base.dart` (matching the Python app's shape) but nothing ever populates it, so generation only ever sees a single note's own body. Worth porting once there's a folder structure large enough to make cross-note linking actually useful.
- `.xlsx` **import isn't implemented**, despite being mentioned in early planning. No note content is naturally spreadsheet-shaped, so this was deprioritized rather than forced. `.txt` / `.md` / `.pdf` / `.docx` / `.pptx` all work.

Not yet built, no fundamental blocker

- **Automatic on-open question generation.** The plan was: check which notes are due for new questions when the app opens, generate without a manual button press. Right now, generation is always a manual "Generate" tap in the note editor.
- **Deeper stats.** The Today tab shows what's due, reviews done today, average recall and a 7-day forecast. There's no per-folder or per-tag accuracy, streak or history chart yet, though `question_history` and `problem_attempts` record everything needed.
- **Personalised FSRS weights.** Scheduling uses FSRS's default 21 parameters. FSRS can optimise them from a user's own review log (a few hundred reviews in); that would need an optimiser, which the `fsrs` Dart package doesn't include.
- **Linking problems to notes or folders.** Problems are a flat list, filterable by status, difficulty, tag and text, and not attached to a note or subject folder.
- **Soft-deleting a question.** The `active` column and the DB-level capability exist (see `database_schema.md`), but no screen exposes a way to deactivate a single question without deleting its entire parent note.
- **Quiz-to-note traceability only goes one level.** A question always belongs to exactly one source note (no note-linking, see above), so "which note did this come from" is a direct lookup, not a list. This was an open question in early planning ("should quizzes get independent folder placement, or always inherit their source note's") and was resolved this way: a question inherits its source note's folder implicitly, via `note_id`; there's no independent quiz-folder concept.

Known rough edges

- No dark mode (savepoint.co.nz has none either).
- The theme and fonts come from savepoint.co.nz's live CSS, but the app hasn't been compared side by side with the site on a real device.